

Metaphysical Foundations, General Systems Theory, and Sequential Dynamic Systems over Fragile Networks

System Architecture Invariant Manifest

September 30, 2026

1 Introduction and Ontological Modalities

Following the formalisms of Edward Zalta's Abstract Object Theory and Situation Theory, we isolate three distinct modal world-lines within a single unified processing manifold:

1. **The Historical Immutable World:** Rigidly designated actualities (s_{actual}) frozen in the past.
2. **The Fictional Mutable World:** Abstract objects encoding properties inside an isolated situation domain (s'), completely open to counterfactual planning rewrites.
3. **The Indexical Real World:** The concrete evaluation layer where sovereign actions and executions occur in real time.

2 The DSL Discretization Interface and Continuous Superposition

Continuous environments present an infinite, unsearchable behavior space. We address this challenge by introducing a LISP-like Domain-Specific Language (DSL) that functions as a structural discretization filter. By maintaining alternative trajectories in a continuous superposition field, multiple overlapping constraints are evaluated simultaneously. Stable paths experience constructive interference, while high-error paths undergo destructive phase-cancellation, mirroring the active wave-reflection compensation scripts.

3 The Mesarovic–Takahara Systems Framework

For large-scale fleet deployments ($N = 50$), complete point-to-point state-space tracking becomes computationally intractable. We address this by formalizing the system under a Mesarovic–Takahara input-output abstract response mapping:

$$S : M \times U \rightarrow Y \quad (1)$$

where U is the Disturbance Set generated by the non-linear Navier-Stokes weather advection field, M is the Control Input Set of discrete structural graph interventions executed by the LISP DSL controller, and Y is the System Outcome Space tracking the layered physical, communication, and computational graph networks ($\mathcal{G}_P, \mathcal{G}_C, \mathcal{G}_{\text{Comp}}$).

Because deterministic controllability fails when communication edges snap over a 100-mile domain, controllability is evaluated over a Performance Target Subset $V' \subset V$. The hypergraph system is completely controllable relative to the performance target if and only if:

$$(\forall u) \left[u \in U \rightarrow (\exists m) (g(m, u) \in V') \right] \quad (2)$$

This establishes that for every accidental environmental breakdown (u), there exists a structural graph modification (m) that keeps the operational service yield inside the safe, high-performing zone.

4 Sequential Dynamic Systems and Forbidden Minors

The 50-drone swarm operates as a Sequential Dynamic System (SDS) where the communication graph topology $\mathcal{G}_C$ is fragile and dynamic.

4.1 The 30-Mile Edge-Snapping Constraint

By imposing a strict 30-mile line-of-sight communication radius boundary, edges are allowed to snap violently as vehicles disperse at high velocities. To maintain network torso cohesion without statistics, the central DSL controller executes real-time graph-minor contractions ($\mathcal{G}/e$). This dynamically reduces local vehicle clusters into unified macro-nodes, compressing the continuous state tracking space.

4.2 The Forbidden Failure Minor ($\mathcal{M}_{\text{danger}}$)

The system’s control boundary is strictly governed by the exclusion of the Forbidden Minor $\mathcal{M}_{\text{danger}}$. We partition the vertex set into active assets $\mathcal{V}_F$ and fuel/power depleted assets $\mathcal{V}_D$. The forbidden minor emerges if the contraction of active communication links leaves an isolated or disconnected topology within the dead node cluster:

$$\mathcal{M}_{\text{danger}} = \mathcal{G} \setminus \mathcal{V}_F / \{(v_i, v_j) \in \mathcal{E} \mid v_i, v_j \in \mathcal{V}_D\} \quad (3)$$

If $\mathcal{M}_{\text{danger}} \prec \mathcal{G}$, the algebraic connectivity collapses to zero, signaling an irreversible network split.

4.3 Reversible Schedules via Self-Similar Groups

To bridge the non-ergodic statistical limits of LISP, a Self-Similar Group Automaton is deployed in the background knowledge base (`kb.pl`). The history of graph state transformations is recorded as an algebraic word. If a locating node becomes isolated on the edge of the 30-mile topology, its state is preserved under a transactional `pending_broadcast` packet. If a terminal asset collapse occurs, the self-similar group utilizes its recursive group generator inversions to reverse the schedule—rolling back task configurations across neighbors transitivity without losing the accumulated target search clues.